

Experimenting With Machine Learning Using Diabetes Datasets

Wyatt McDonnell¹, Hongbiao Zeng, PhD²

¹Department of Computer Science, Fort Hays State University



FORT HAYS STATE
UNIVERSITY

Abstract

The purpose of this research is to understand how to implement machine learning in a practical scenario. There were two diabetes datasets^{[5][6]} used for testing the machine learning models. These datasets contain information relevant to a person’s health, as well as whether that subject had diabetes. I used a total of four models, and three of those models were manually programmed. The model which was not manually programmed was used for comparison with a similar model. This research directly compares and shows the factors which affect the efficiency of each machine learning model.

Introduction

There are two machine learning algorithms used in my models are a feedforward neural network (FNN) and a support vector machine. A feedforward neural network processes information in a single direction. The initial data is received by an input layer. Then, this data is passed through any number of hidden layers which modify the data with weights and biases. After this, the output layer produces an output relevant to the data provided^[2]. A support vector machine (SVM) is a machine learning model which separates data into different classes based on whether it is similar enough to the other data^[4]. The learning rate of each model affects the size at which the weights and biases adjust. The type of output my models produce is a prediction of whether a person has diabetes.

Methods

First, the classifications for the data of two datasets were transferred to integers. Then, the bias of the first dataset was removed by trimming the amount of data with an output for people with no diabetes until it was the outputs were equal in number. Both of the datasets' data order was randomized. The first dataset is separated into two datasets for testing and training. The second dataset is only used for testing. The setup is the same for every model. Every time there was a test done by each model, the data order is initially randomized. Every single model was run on Google Collab and coded in Python. There were separate projects made for each manually created machine learning model. The models I created are modified versions of a feedforward neural network^[7] and a support vector machine^[1] programmed by other people. The first model I programmed was a feedforward neural network. This model had one input, hidden, and output layer. My model was given a learning rate of 0.01. The second model I programmed was a support vector machine. This model used a linear classifications, and it had a learning rate of 0.001. In the same project, code for a linear support vector machine from the sklearn library was used for comparison^[3]. The third model I programmed was an improved version of the first model’s feedforward neural network. In this model, there was one input, four hidden layers, and one output layer. The learning rate was the same as the first model.

Analysis

The accuracy of a machine learning model depends on the data it uses and improvements done to the model itself. My first and third models were FNN, but they required a different number of iterations to improve. The accuracy of my first model took around 36,000 iterations to reach a plateau for improvement, but my third model took around 4,450 iterations to reach a plateau of improvement. This is due to the increase in layers of the third model. It should be noted that the test data has a plateau in improvement earlier than the training data. In the second model my SVM took longer to run than the sklearn, but it was more accurate than the sklearn in fewer iterations. The code for sklearn is more efficient than my second model’s code because it can run thousands more iterations in just a few seconds compared to the minutes it would take my second model to run the same number of iterations. Based on the data, it can be concluded that feedforward neural networks are the best model to use. This is supported by the fact that there were various data types in the datasets which seem somewhat disconnected in relation to whether someone has diabetes. A feedforward neural network works similar to a brain, so it can make useful connections and remove any unhelpful connections.

Results

For every single model there were separate tests done for each number of iterations in the models’ tables. Accuracy of my first model:
The accuracy improvements for this model on the training data began plateauing around 36,000 iterations.

Iterations	First Dataset	Second Dataset
10,000	78.45%	62.20%
25,000	85.71%	76.71%
50,000	88.00%	81.18%

Accuracy of my second model:

Iterations	First Dataset	Second Dataset
1,000	81.94%	90.00%
1,500	83.67%	84.28%
2,000	84.27%	83.76%
2,500	77.54%	59.84%
5,000	81.94%	88.65%

Accuracy of my third model:

The accuracy improvements for this model on the training data began plateauing around 4,450 iterations.

Iterations	First Dataset	Second Dataset
1,000	75.24%	85.51%
1,500	77.80%	85.87%
2,000	80.13%	68.24%
2,500	74.18%	56.36%
5,000	79.80%	67.43%

```
[ ] def test_neuralnetwork_accuracy(test_X, test_Y):
    #initialize the variable for correct predictions
    correct = 0
    #set the amount of data to the size of the dataset
    tested_data = len(test_X[0])

    #compare the prediction of the data to the actual data
    for i in range(0, tested_data):
        prediction, has_diabetes = test_prediction(test_X, test_Y, i, W1, b1, W2, b2, W3, b3, W4, b4, W5, b5)
        #if the prediction is correct add 1 to the count of correct predictions
        if(prediction == has_diabetes):
            correct += 1
    #print the accuracy of the test data
    print("Accuracy on test data: " + str(correct/tested_data))
```

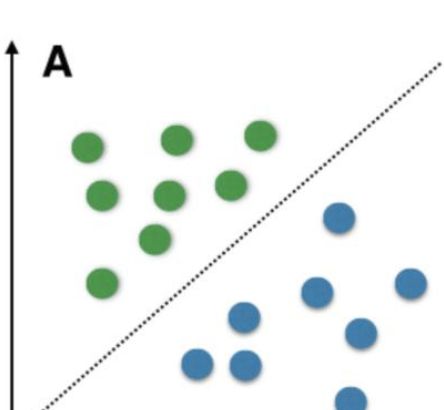
Code for computing the accuracy of the first and third model

```
[ ] def accuracy(y_true, y_pred):
    accuracy = np.sum(y_true == y_pred) / len(y_true)
    return accuracy
```

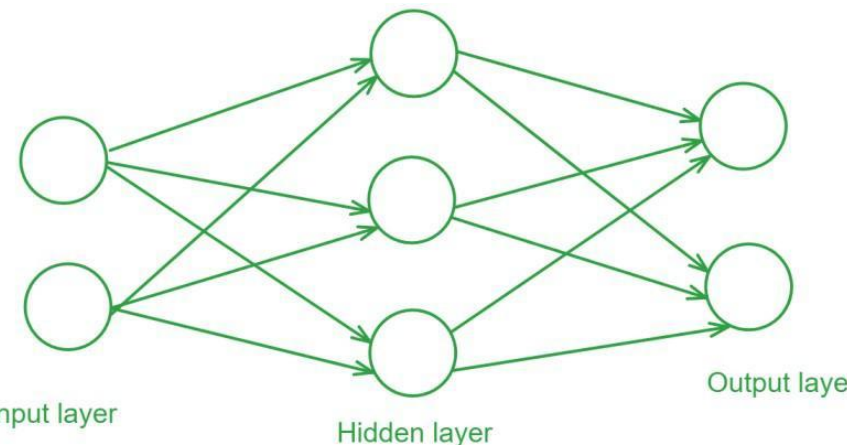
Code for computing the accuracy of the second and sklearn model

Conclusion

The information gathered shows that the machine learning model that should be used depends on the dataset. The accuracy improvements found in the results section support this idea. Some of my models improved in less iterations than others. It was easier for some models to make these improvement because the data had connections that were non-linear. The support vector models had difficulties improving because the connections they needed to make were non-linear. The feedforward neural networks did not struggle with the same issue due to their structure being similar to a brain. The models could not get an accuracy above 90%, and this is most likely due to the dataset sections being given to each model only containing the data from 6,000 people. In conclusion, the accuracy of a machine learning model in practical scenarios depends on the type of data given, the amount of data given, and what the model is used for.



visualization of how a SVM works [4]



visualization of how a FNN works [2]

References

1. AssemblyAI Community. (2022). *AssemblyAI-Community/Machine-Learning-From-Scratch*. GitHub. [https://github.com/AssemblyAI-Community/Machine-Learning-From-Scratch/blob/main/09 SVM/svm.py](https://github.com/AssemblyAI-Community/Machine-Learning-From-Scratch/blob/main/09%20SVM/svm.py)
2. GeeksforGeeks. (2024, June 20). *Feedforward neural network*. <https://www.geeksforgeeks.org/feedforward-neural-network/>
3. GeeksforGeeks. (2025, Jan. 30). *Implementing SVM from Scratch in Python*. <https://www.geeksforgeeks.org/implementing-svm-from-scratch-in-python/>
4. GeeksforGeeks. (2020, June 9). *Introduction to Support Vector Machines (SVM)*. <https://www.geeksforgeeks.org/introduction-to-support-vector-machines-svm/>
5. Mustafa, M. (2023). *Diabetes prediction dataset*. Kaggle. <https://www.kaggle.com/datasets/iammustafatz/diabetes-prediction-dataset>
6. Parvizi, A. M. (2024). *Diabetes Prediction DataSet*. Kaggle. <https://www.kaggle.com/datasets/amirmohammadparvizi/diabetes-prediction-dataset>
7. Zhang, S. [Samson Zhang]. (2020, Nov. 24). *Building a neural network FROM SCRATCH (no Tensorflow/Pytorch, just numpy & math)* [Video]. YouTube. <https://youtu.be/w8yWXqWQYmU>

Acknowledgments

This student research would not have been possible without the guidance of professor Hongbiao Zeng. This professor taught the basics of machine learning and provided guidance in learning projects. The student was able to apply what he learned in this poster.



image of professor
Hongbiao Zeng